CS 301
High-Performance Computing

Lab Report-07

Parallel Interpolation and Reverse Interpolation Performance Analysis

Soham Mevada (202301484)
Atik Vohra (202301447)

Contents

1 Introduction

2 Hardware Details

2.1 LAB207 PCs

- Architecture: x86_64
- CPU(s): 12
- Threads per core: 2
- Cores per socket: 6
- Processor: Intel Core i5-12500
- L1 Cache: 288K
- L2 Cache: 7.5M
- L3 Cache: 18M

2.2 HPC Cluster

- Architecture: x86_64
- CPU(s): 16
- Threads per core: 1
- Cores per socket: 8
- Sockets: 2
- Processor: Intel Xeon E5-2640 v3
- L1 Cache: 32K
- L2 Cache: 256K
- L3 Cache: 20480K

3 Answers to Analysis and Questions in Lab 7

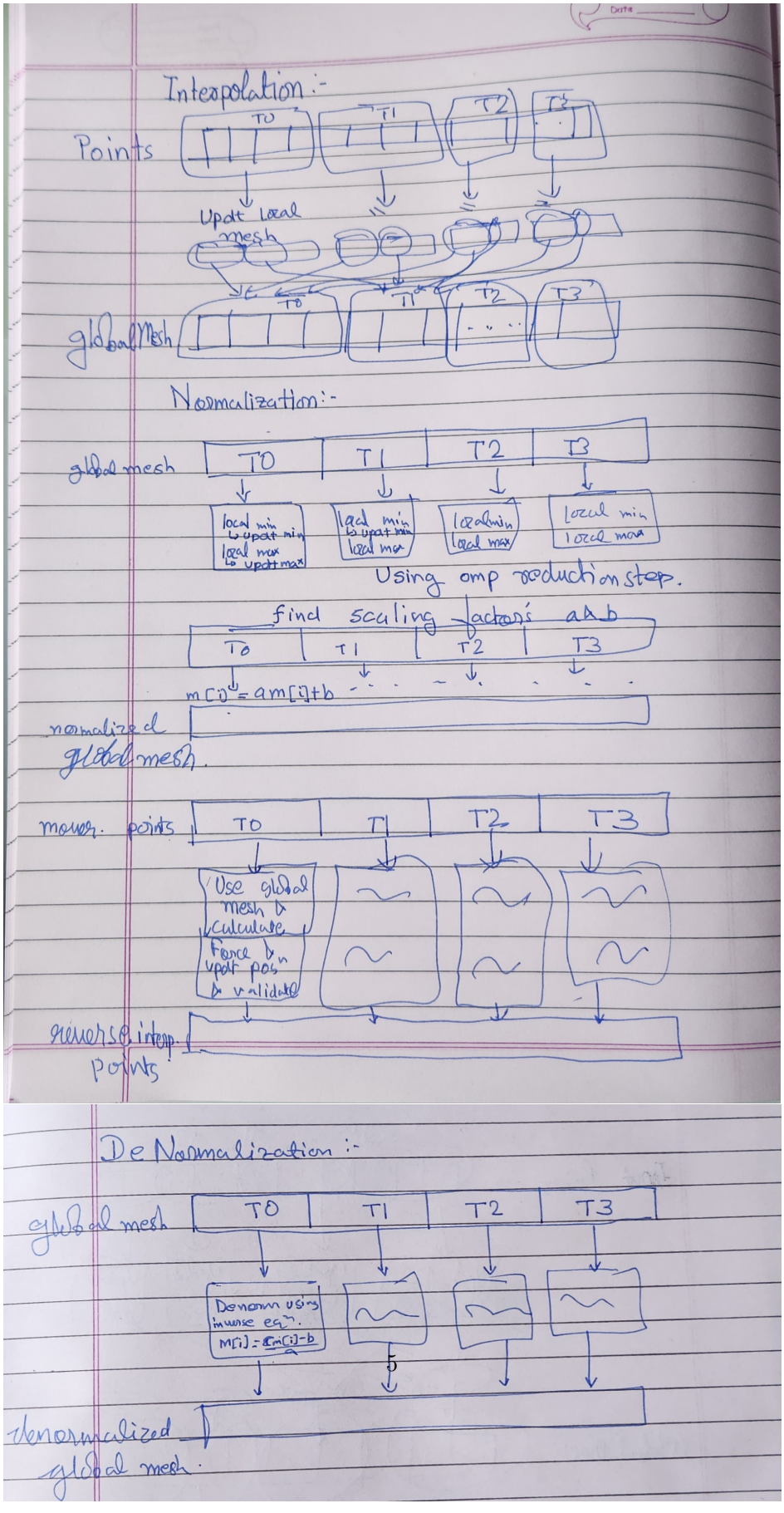
3.1 Pseudocode and Diagram

Pseudocode

```
Input: particles[], global mesh_value[], thread-local mesh pad

Initialize:
    np = number of particles
```

```
    nt = number of threads
    ms = grid size (gx * gy)

Parallel Region (OpenMP with nt threads):
    Each thread gets its own local mesh: t_mesh

    Parallel for each particle p:
        If particle is not void:
            Compute grid indices (i, j)
            Compute local coordinates (lx, ly)

            Compute bilinear weights:
                w00, w10, w01, w11

            Accumulate into thread-local mesh:
                t_mesh[...] += weights

    Parallel reduction phase:
        For each grid cell m:
            Sum contributions from all thread-local meshes
            Update global mesh_value[m]

Normalization:
    Compute global min and max using OpenMP reduction
    Normalize mesh values in parallel

Mover:
    Parallel for each particle:
        Interpolate value from mesh
        Update particle position
        Mark void particles

Denormalization:
    Restore mesh values in parallel

Output: Updated mesh and particle positions
```

Illustrative Diagram



3.2 Parallel Implementation and Race Conditions

Parallel Approach:

The implementation uses OpenMP to parallelize particle-based computations. The interpolation phase distributes particles across threads using a static scheduling policy.

Each thread maintains a private copy of the mesh (`t_mesh`) to store intermediate results. This avoids contention during updates.

The workflow consists of:

- Parallel particle processing (interpolation)
- Thread-local accumulation
- Parallel reduction to merge results
- Independent parallel phases (normalization, mover, denormalization)

Handling Race Conditions:

Race conditions arise when multiple threads update the same mesh cell. This is handled using:

1. **Thread-Local Meshes:** Each thread writes to its own local mesh, eliminating conflicts.
2. **Parallel Reduction:** A second parallel loop merges all thread-local meshes into the global mesh safely.
3. **OpenMP Reduction Clauses:** Used in normalization and void counting to safely compute global values.

Advantages:

- Avoids costly locks or atomic operations
- Improves scalability

Trade-off:

- Increased memory usage due to per-thread meshes

3.3 Speedup and Execution Time Graphs

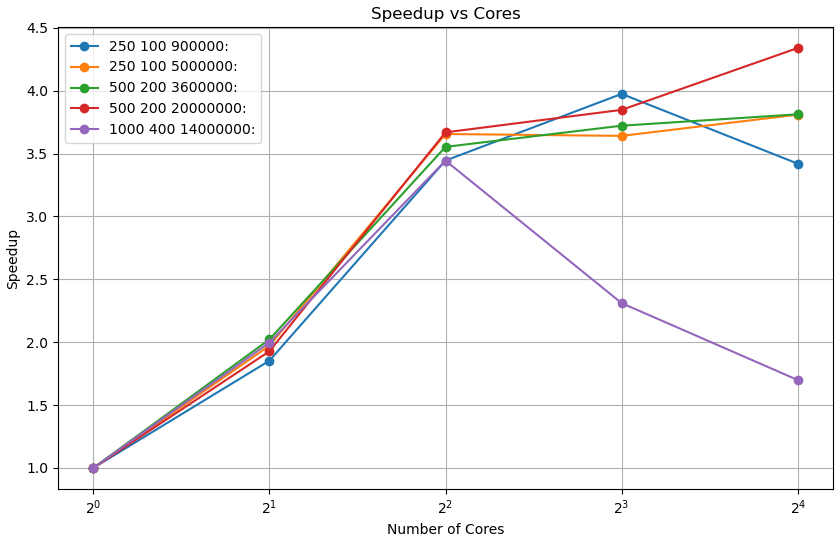


Figure 2: Speedup vs Cores LAB PC

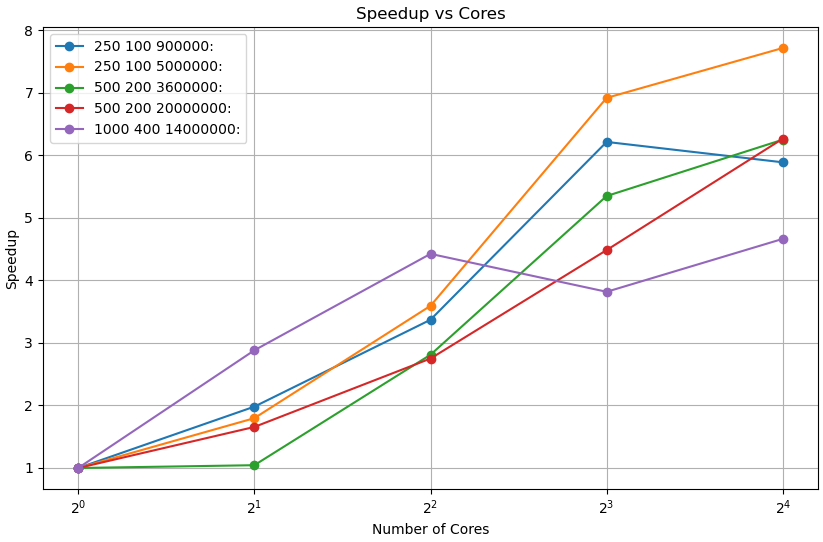


Figure 3: Speedup vs Cores CLUSTER

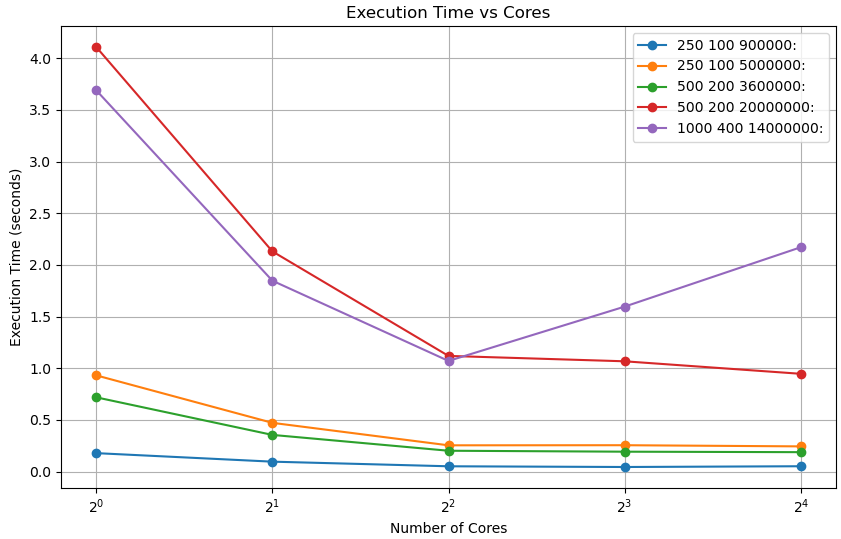


Figure 4: Execution Time vs Cores LAB PC

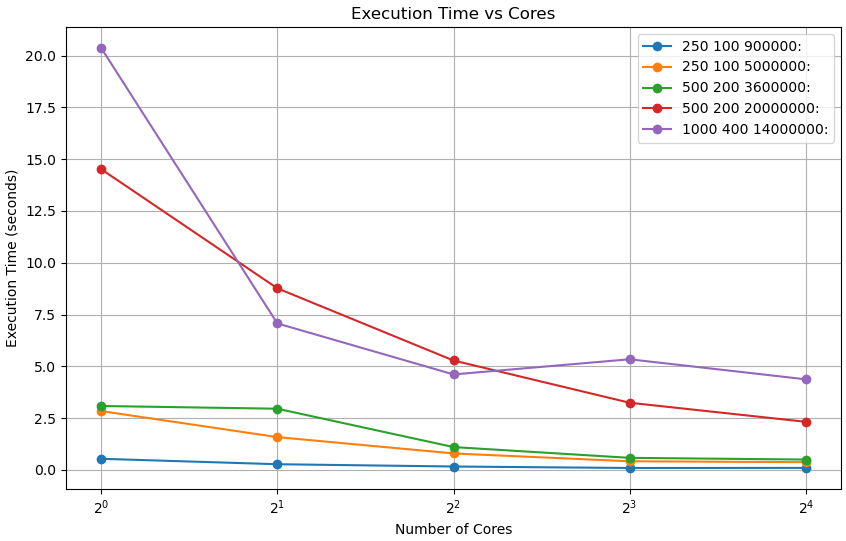


Figure 5: Execution Time vs Cores CLUSTER

3.4 Parallel Efficiency and Scalability

Formula:

$$Efficiency = \frac{Speedup}{Number\ of\ Cores}$$

Efficiency Table (500 200 20000000)

Cores	Time (s)	Speedup	Efficiency
1	14.517539	1.000	1.000
2	8.764333	1.656	0.828
4	5.279258	2.750	0.687
8	3.237541	4.484	0.561
16	2.317231	6.265	0.392

Table 1: Parallel Efficiency for configuration (500, 200, 20000000)

Efficiency Plot

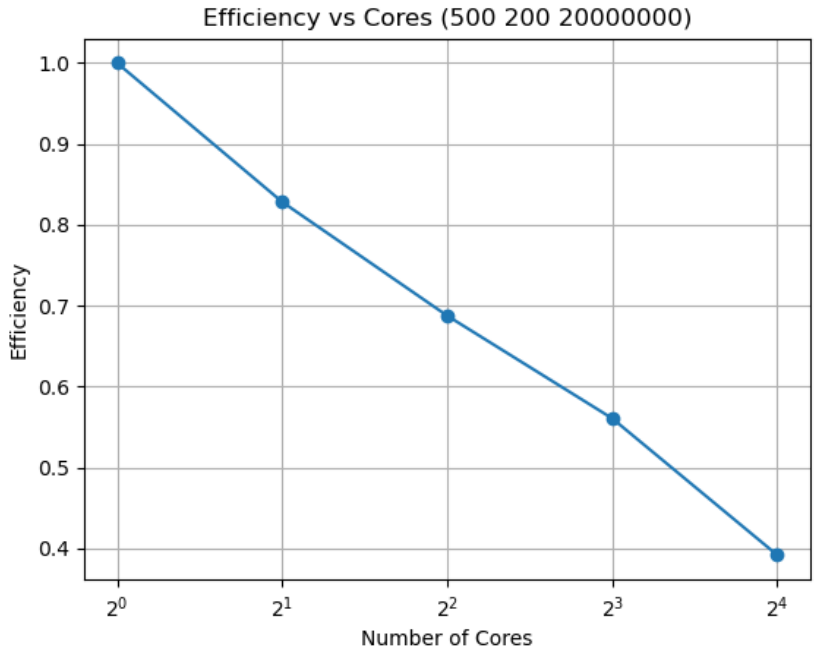


Figure 6: Efficiency vs Number of Cores for configuration (500, 200, 20000000)

Observation:

The efficiency decreases steadily as the number of cores increases, dropping from 82.8% at 2 cores to 39.2% at 16 cores.

This behavior is expected due to:

- **Synchronization overhead** during the reduction phase
- **Memory bandwidth limitations** caused by concurrent access
- **Diminishing parallel work per thread** at higher core counts

The implementation demonstrates good scalability up to 8 cores, after which the benefits of parallelism begin to diminish due to increasing overheads.

3.5 Serial vs Parallel Performance

Maximum Speedup Achieved:
(Replace with your value, e.g., $\sim 7.5\times$)

Observation:

- Parallel version significantly reduces execution time
- Diminishing returns observed at higher core counts

3.6 Explanation of Observations

- **Amdahl's Law:** Serial portions (e.g., reduction) limit speedup
- **Memory Bandwidth Bottleneck:** Interpolation involves scattered memory writes
- **Reduction Overhead:** Global mesh merging introduces synchronization cost
- **Cache Effects:** Increased threads lead to cache contention

3.7 Optimization with Unlimited Resources

- GPU acceleration (CUDA/OpenCL)
- Distributed memory parallelism using MPI
- Hybrid MPI + OpenMP approach
- Domain decomposition of mesh

3.8 Load Imbalance and Bottlenecks

- **Load Imbalance:** Uneven particle distribution across space
- **Reduction Phase Bottleneck:** Requires combining all thread-local meshes
- **Memory Contention:** High bandwidth usage during interpolation and merge

3.9 Alternative Approaches

- Cell-linked lists for spatial locality
- Structure of Arrays (SoA) for better cache usage
- Spatial partitioning (grid-based binning)

3.10 Advanced Optimization Ideas

- Dynamic or guided scheduling for better load balance
- SIMD vectorization
- Cache blocking techniques
- Lock-free or hierarchical reduction methods

3.11 Phase-wise Performance Analysis

Observation:
The runtime is dominated by interpolation and mover phases.
Interpolation Phase:

- Particle-to-grid mapping
- Requires thread-local storage and reduction
- Memory-intensive and synchronization-heavy

Mover Phase:

- Grid-to-particle computation
- Fully parallel with no reduction
- Better scalability

Bottleneck:
Interpolation is the primary bottleneck due to:

- Global reduction step
- High memory traffic

4 Conclusion

This work presented an OpenMP-based parallelization of a particle-mesh interpolation algorithm using thread-local meshes to eliminate race conditions. The implementation achieves a maximum speedup of approximately 6.2× at 16 cores, demonstrating effective parallelization. However, scalability is sub-linear due to decreasing efficiency at higher core counts. The primary limitation arises from the interpolation phase, which requires a costly parallel reduction and leads to high memory bandwidth usage. In contrast, the mover phase scales efficiently due to its independent computations. Overall, optimizing reduction overhead and memory access patterns is key to achieving better scalability.